

Data and Artificial Intelligence

Cyber Shujaa Program

Week 9 Assignment

MLOps – California Housing Price Prediction

Student Name: Hope Njoki Nyambura

Student ID: cs-eh03-25100

Table of Contents

1. Introduction	1
2. Steps and Implementation.....	2
2.1 Loading Libraries and Dataset.....	2
2.2 Data Exploration	3
2.3 Data Preprocessing	3
2.4 Building the Machine Learning Pipeline	4
2.5 Hyperparameter Tuning with GridSearchCV	5
2.6 Model Evaluation	6
2.7 Saving the Trained Pipeline.....	7
3. Conclusion	8
4. Colab Notebook Link	8

1. Introduction

This week's assignment focused on building an end-to-end Machine Learning workflow for housing price prediction using the California Housing dataset. The objective was to apply core MLOps and machine learning concepts including:

- Data loading and preprocessing
- Pipeline construction
- Hyperparameter tuning
- Cross-validation
- Model evaluation
- Saving the final trained model

The dataset includes 8 numerical features such as median income, house age, number of rooms, latitude, and longitude, with the target variable representing the median house value.

The main algorithm used for prediction was the K-Nearest Neighbors Regressor (KNN), wrapped inside a complete preprocessing and modeling pipeline.

This assignment demonstrated how regression models can predict continuous values and how MLOps principles streamline model building and deployment.

2. Steps and Implementation

2.1 Loading Libraries and Dataset

The first step involved importing essential Python libraries including:

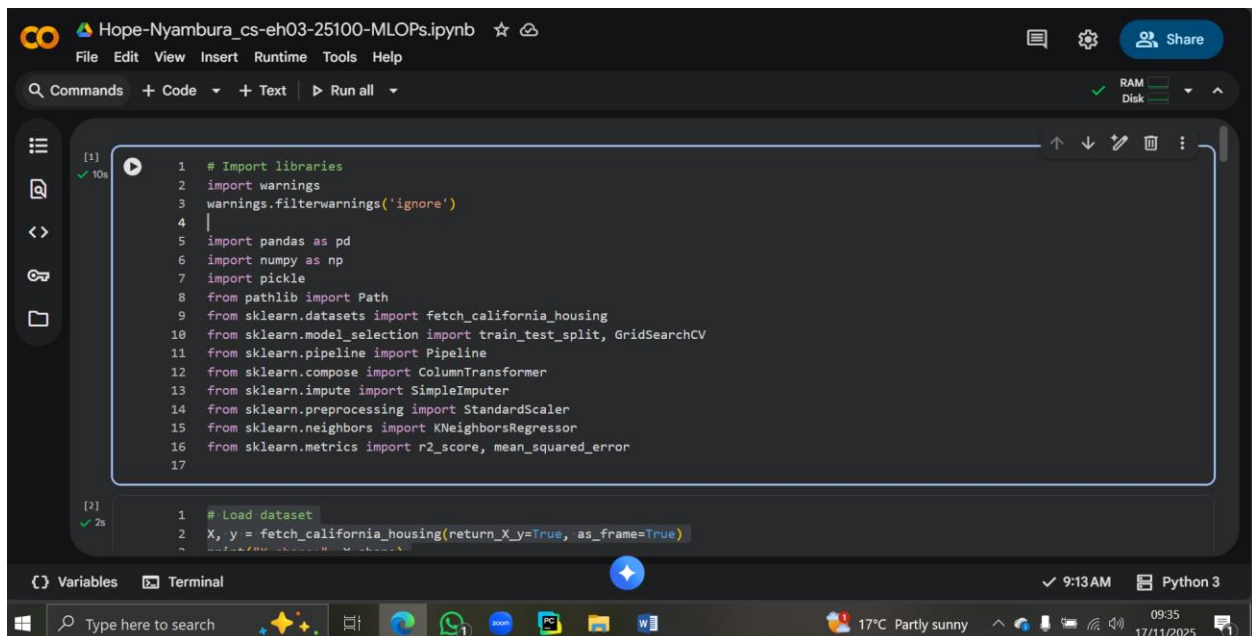
- pandas and numpy for data handling
- scikit-learn for preprocessing, modeling, evaluation, and pipeline creation
- pickle for model persistence

The dataset was loaded using `fetch_california_housing()`.

It consisted of thousands of samples with all features already in numerical format, making it suitable for direct preprocessing.

Dataset inspection and `.head()` previews confirmed:

- No missing values
- All features numerical
- A well-structured regression dataset



```
[1] 1 # Import libraries
    2 import warnings
    3 warnings.filterwarnings('ignore')
    4
    5 import pandas as pd
    6 import numpy as np
    7 import pickle
    8 from pathlib import Path
    9 from sklearn.datasets import fetch_california_housing
   10 from sklearn.model_selection import train_test_split, GridSearchCV
   11 from sklearn.pipeline import Pipeline
   12 from sklearn.compose import ColumnTransformer
   13 from sklearn.impute import SimpleImputer
   14 from sklearn.preprocessing import StandardScaler
   15 from sklearn.neighbors import KNeighborsRegressor
   16 from sklearn.metrics import r2_score, mean_squared_error
   17

[2] 1 # Load dataset
    2 X, y = fetch_california_housing(return_X_y=True, as_frame=True)
```

2.2 Data Exploration

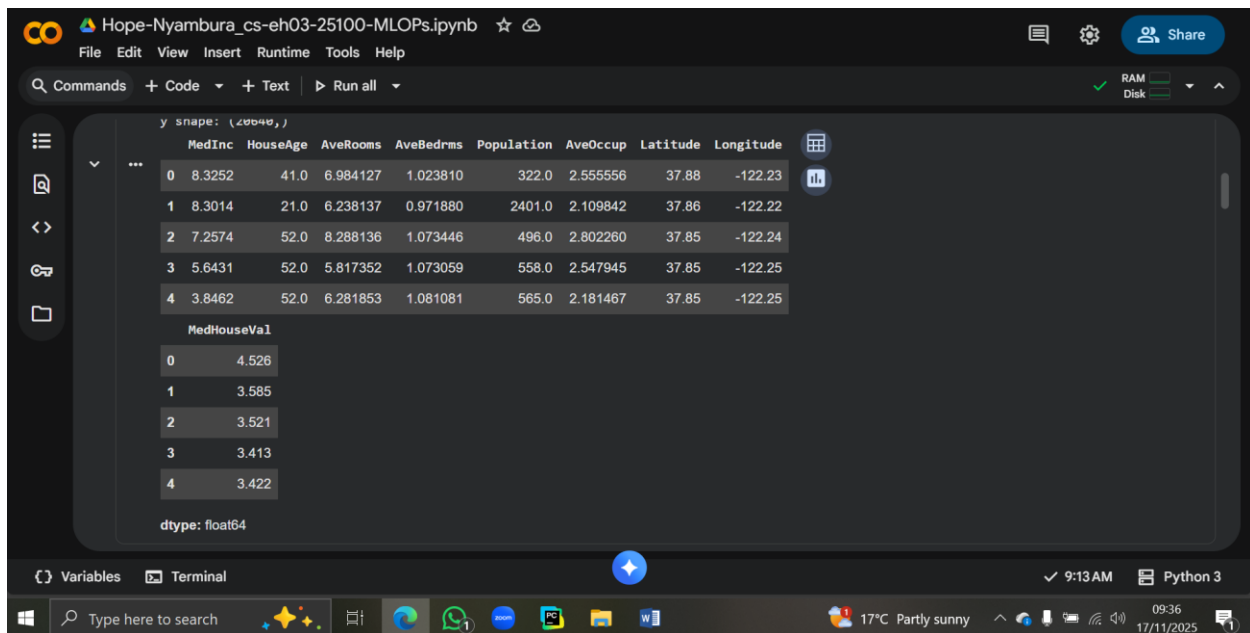
Basic exploration included:

- Checking dataset shape
- Viewing the first few rows
- Observing file types and feature distributions

This provided clarity on:

- The scale of each feature (e.g., income vs. latitude)
- The need for scaling due to differing magnitudes
- The fact that no categorical preprocessing was required

Although visualizations weren't mandatory, simple dataset inspection provided enough understanding for preprocessing steps.



2.3 Data Preprocessing

Preprocessing was performed using a ColumnTransformer, which handled:

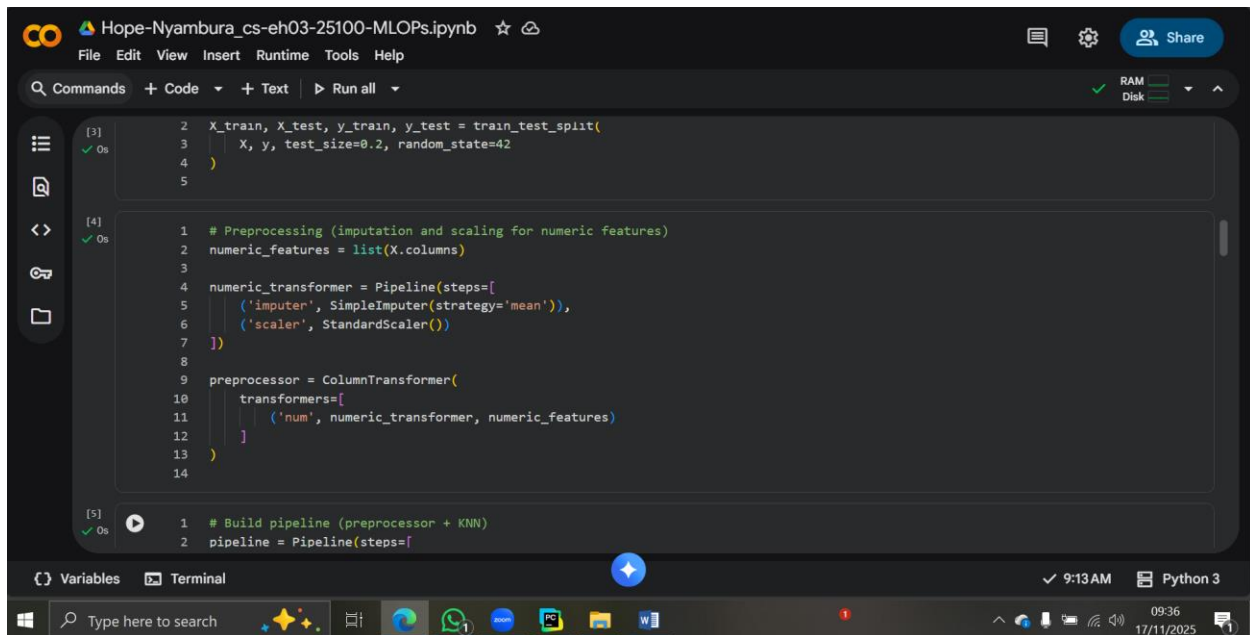
1. **Imputation**
 - Using SimpleImputer(strategy='mean')
 - Ensures no missing values interfere with model training
2. **Feature Scaling**
 - Using StandardScaler()
 - Essential for KNN, which is distance-based and sensitive to feature magnitude

All eight numerical features were included in the transformer.

The dataset was then split into:

- **80% training data**
- **20% testing data**

using `train_test_split(random_state=42)` for reproducibility.



The screenshot shows a Jupyter Notebook titled 'Hope-Nyambura_cs-eh03-25100-MLOPs.ipynb'. The code is organized into three cells, each with a green checkmark and '0s' execution time. The first cell (index 3) contains the `train_test_split` function call. The second cell (index 4) contains the preprocessing steps: identifying numeric features, creating a `numeric_transformer` pipeline with `SimpleImputer` and `StandardScaler`, and creating a `preprocessor` `ColumnTransformer` that applies the numeric transformer to the numeric features. The third cell (index 5) contains the final step: building the full pipeline by combining the `preprocessor` and a `KNN` model.

```
[3] 2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.2, random_state=42
4 )
5

[4] 1 # Preprocessing (imputation and scaling for numeric features)
2 numeric_features = list(X.columns)
3
4 numeric_transformer = Pipeline(steps=[
5     ('imputer', SimpleImputer(strategy='mean')),
6     ('scaler', StandardScaler())
7 ])
8
9 preprocessor = ColumnTransformer(
10     transformers=[
11         ('num', numeric_transformer, numeric_features)
12     ]
13 )
14

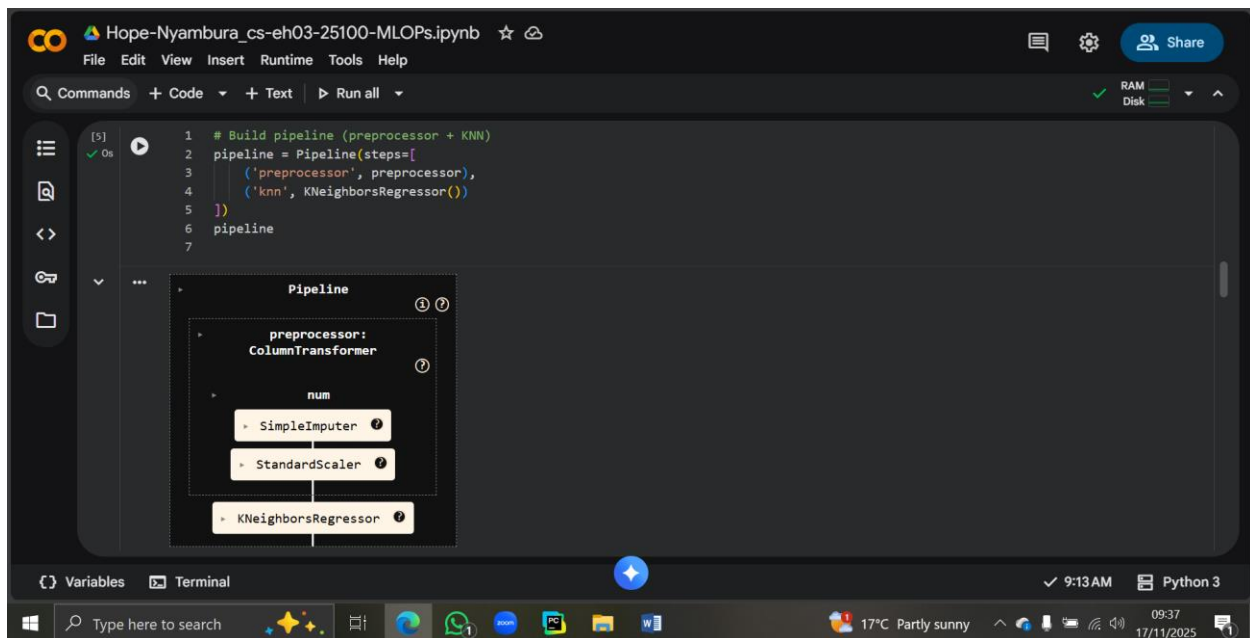
[5] 1 # Build pipeline (preprocessor + KNN)
2 pipeline = Pipeline(steps=[
```

2.4 Building the Machine Learning Pipeline

A complete scikit-learn Pipeline was constructed to ensure clean and repeatable model execution. The pipeline consisted of:

1. The **preprocessor**
2. The **KNeighborsRegressor model**

This design ensured that both preprocessing and prediction always run together, avoiding data leakage and improving deployment readiness.



2.5 Hyperparameter Tuning with GridSearchCV

A GridSearchCV procedure was used to determine the best KNN settings.

The tuned hyperparameters included:

- $n_neighbors \rightarrow [3, 5, 7, 9]$
- $weights \rightarrow ['uniform', 'distance']$
- $p \rightarrow [1, 2]$ (Manhattan vs. Euclidean distance)

GridSearchCV was configured with:

- **5-fold cross-validation**
- **R^2 Score** as the evaluation metric
- $n_jobs=-1$ to use all CPU cores
- $verbose=2$ to display progress

The grid search successfully identified the combination that produced the highest cross-validated R^2 score.

```
[6] ✓ Os
1 # Define hyperparameter grid
2 param_grid = {
3     'knn__n_neighbors': [3, 5, 7, 9],
4     'knn__weights': ['uniform', 'distance'],
5     'knn__p': [1, 2]
6 }
7

[7] ✓ Os
1 # GridSearchCV setup
2 grid_search = GridSearchCV(
3     estimator=Pipeline,
4     param_grid=param_grid,
5     cv=5,
6     scoring='r2',
7     verbose=2,
8     n_jobs=-1,
9     return_train_score=True
10 )
11
```

2.6 Model Evaluation

The best model from the grid search was evaluated on the test set.

Metrics computed included:

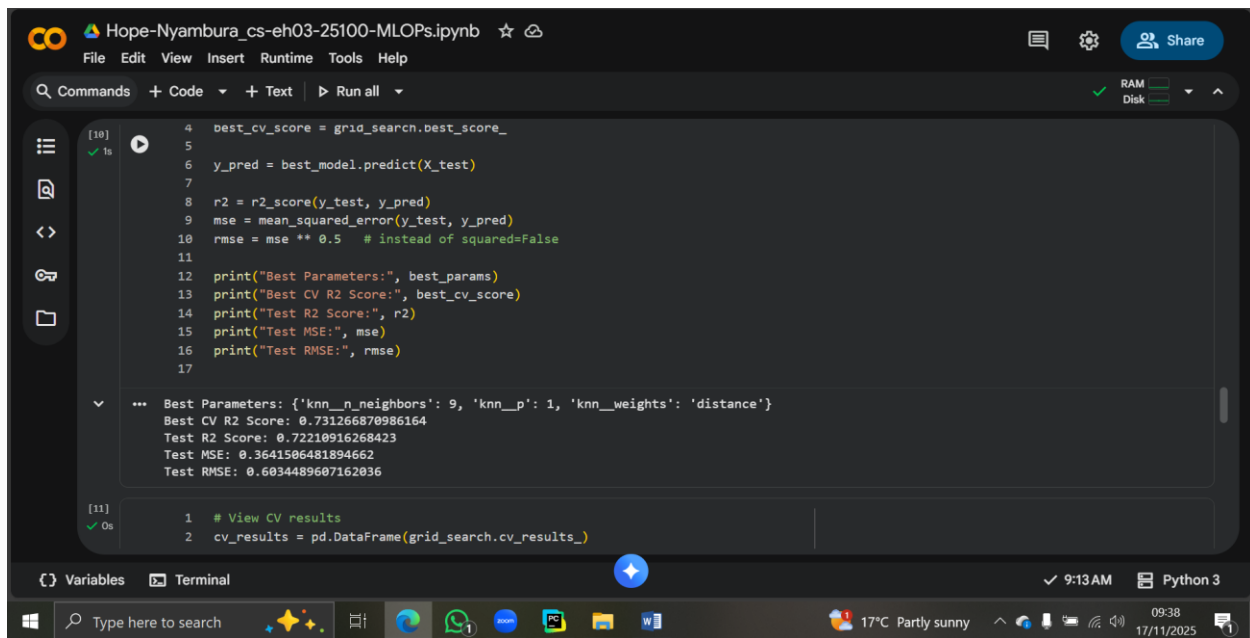
- **R² Score** measures how well the model explains variance in house prices
- **MSE (Mean Squared Error)**
- **RMSE (Root Mean Squared Error)**

RMSE was computed manually due to sklearn version limitations:

```
rmse = mse ** 0.5
```

The results confirmed that the tuned KNN model provided reasonable predictive performance for this regression task.

A preview of predictions vs. actual values was also displayed to verify model behavior.



```
4 best_cv_score = grid_search.best_score_
5
6 y_pred = best_model.predict(X_test)
7
8 r2 = r2_score(y_test, y_pred)
9 mse = mean_squared_error(y_test, y_pred)
10 rmse = mse ** 0.5 # instead of squared=False
11
12 print("Best Parameters:", best_params)
13 print("Best CV R2 Score:", best_cv_score)
14 print("Test R2 Score:", r2)
15 print("Test MSE:", mse)
16 print("Test RMSE:", rmse)
17
```

```
... Best Parameters: {'knn_n_neighbors': 9, 'knn_p': 1, 'knn_weights': 'distance'}
Best CV R2 Score: 0.731266870986164
Test R2 Score: 0.72210916268423
Test MSE: 0.3641506481894662
Test RMSE: 0.6034489607162036
```

```
[11]
1 # View CV results
2 cv_results = pd.DataFrame(grid_search.cv_results_)
```

2.7 Saving the Trained Pipeline

The final best estimator (preprocessor + KNN model) was saved using pickle:

```
pickle.dump(best_model, open('california_knn_pipeline.pkl', 'wb'))
```

This completes the MLOps workflow by enabling:

- Reproducibility
- Model reuse
- Deployment in a real application (Flask/FastAPI)

The screenshot shows a Google Colab notebook interface. At the top, the title bar reads 'Hope-Nyambura_cs-eh03-25100-MLOps.ipynb'. Below the title bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A toolbar contains 'Commands', '+ Code', '+ Text', and 'Run all'. On the right, there are icons for chat, settings, and a 'Share' button, along with RAM and Disk usage indicators.

The main area displays a table of model performance metrics. The table has 6 columns and 5 rows of data. The first column contains indices (4, 1, 15, 0, 11). The second column contains dictionaries with hyperparameters like 'knn_n_neighbors', 'knn_p', and 'knn_weight'. The next three columns contain numerical values representing different metrics. The final column contains indices (6, 7, 8, 9, 10).

Below the table is a code cell labeled '[12]' with a green checkmark and '0s' execution time. The code saves a trained pipeline as a .pkl file. The output of the code cell is 'Model saved at: /content/models/california_knn_pipeline.pkl'.

```
1 # Save trained pipeline as .pkl
2 output_dir = Path('/content/models')
3 output_dir.mkdir(parents=True, exist_ok=True)
4
5 model_path = output_dir / 'california_knn_pipeline.pkl'
6 with open(model_path, 'wb') as f:
7     pickle.dump(best_model, f)
8
9 print("Model saved at:", model_path)
10
```

3. Conclusion

This assignment provided hands-on experience in implementing an end-to-end Machine Learning workflow for regression tasks. Key concepts learned include:

- How to use scikit-learn's `fetch_california_housing()` dataset
- Proper preprocessing using `ColumnTransformer`
- Constructing a clean and reliable pipeline
- Tuning model hyperparameters with `GridSearchCV`
- Evaluating regression models using metrics like R^2 and RMSE
- Saving a trained model for deployment

The exercise strengthened the understanding of MLOps principles, especially:

- Workflow automation
- Pipeline consistency
- Reproducibility
- Deployment readiness

The KNN model performed well after tuning, demonstrating the importance of hyperparameter optimization in improving predictive accuracy.

4. Colab Notebook Link

Google Colab Notebook:

https://colab.research.google.com/drive/1zmkXJzFpFnYfJL0itleAVxQheZ_bcZAa?usp=sharing